Password Cracking & Credential Attack Suite

Student Name : Sakshi Maruti Nalawade .

Institute : Unified Mentor

Lab Environment : Kali Linux

INDEX	
	Title
1.	Objective
2.	Tools
3.	Environment
4.	Methodology
5.	Results and Observations

Project Objectives

• Develop a dictionary generator for password testing.
• Extract password hashes from Linux shadow files and Windows SAM database (ethical environment only).
• Build a brute-force simulation engine to test password robustness.
• Analyse password strength based on complexity, entropy, and predictability.
• Generate a detailed audit report on password vulnerabilities and mitigation steps.

Tools & Technologies Used

Programming Languages:
• Python (recommended)
• Bash (optional)

Modules/Tools Used:
• hashlib (hashing operations)
• crypt / passlib (Linux password hash processing)
• reg.exe (Windows hive export)
• Samdump2 (hash extraction)
• John the Ripper / Hashcat (optional reference tools, not mandatory)

Documentation Tools:
• Word / Google Docs
• word (flowcharts and architecture diagrams)

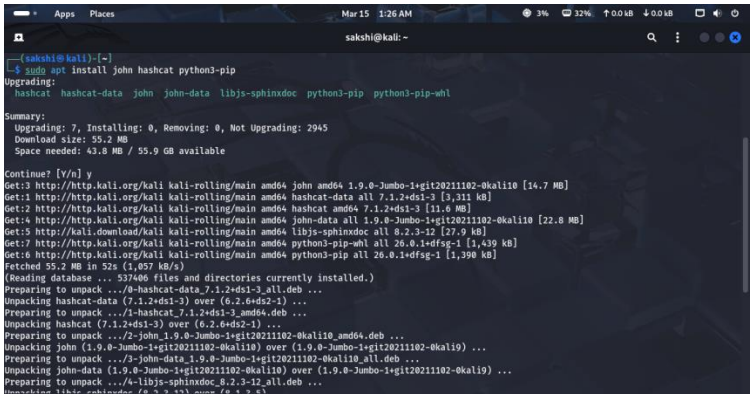
Workflow / Architecture (Practical Explanation)

STEP 1 — Setup Lab Environment

Install required tools in Kali Linux

sudo apt update
sudo apt install john hashcat python3-pip
pip3 install passlib

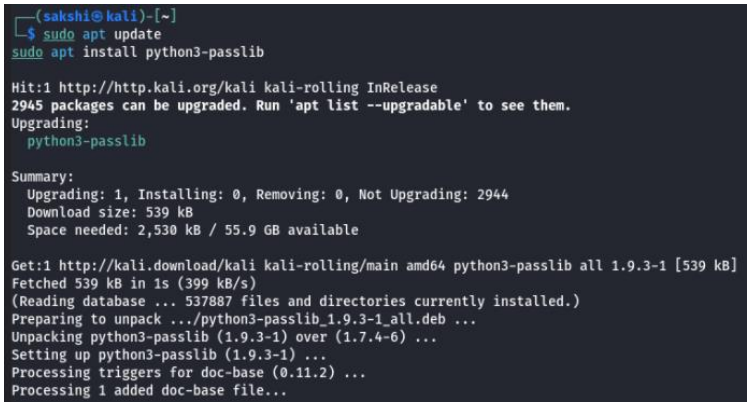
sudo apt update
sudo apt install john hashcat python3-pip



• Use the APT package manager (Recommended for system-wide installation): Check if passlib is available in the Kali Linux repositories and install it using apt, which is the preferred way to install system-wide Python packages.

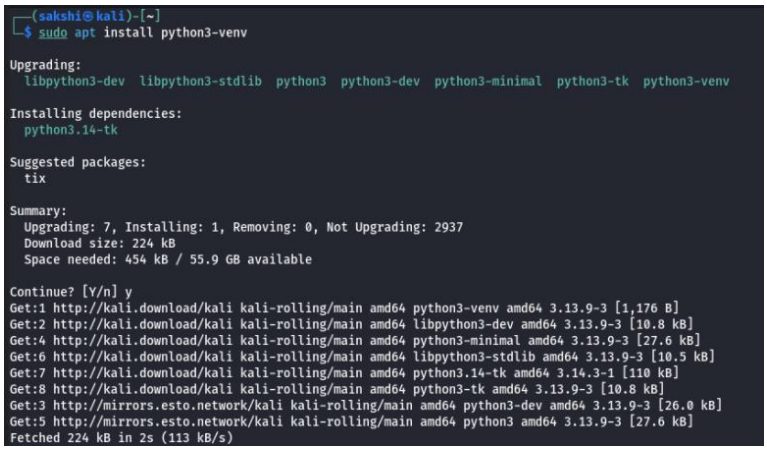
sudo apt update

sudo apt install python3-passlib



• Use a Python virtual environment (Recommended for project-specific use): This is the safest method to install non-Kali-packaged Python libraries, as it isolates the installation from the system's Python environment.

sudo apt install python3-venv



• Create a virtual environment in your project directory (replace venvname with a name of your choice):

python3 -m venv venvname

```
┌──(sakshi㉿kali)-[~]
└─$ python3 -m venv venvname
```

• Activate the virtual environment:
source venvname/bin/activate

```
┌──(sakshi㉿kali)-[~]
└─$ source venvname/bin/activate
```

• Install passlib using pip within the activated environment:
pip install passlib

```
┌──(venvname)─(sakshi㉿kali)-[~]
└─$ pip install passlib

Collecting passlib
  Downloading passlib-1.7.4-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading passlib-1.7.4-py2.py3-none-any.whl (525 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 525.6/525.6 kB 889.8 kB/s  0:00:00
Installing collected packages: passlib
Successfully installed passlib-1.7.4
```

STEP 2 — Dictionary Generator

• Create a Python script to generate password wordlists.
File: dictionary_generator.py

```
sakshi@kali: ~/Desktop/Dictionary_Generator
  GNU nano 8.4                                   dictionary_generator.py *
import itertools

name = "sakshi"
dob = "2001"

words = [name, dob, "password", "admin"]

symbols = ["@", "#", "123", "!", ""]

wordlist = []

for w in words:
    for s in symbols:
        wordlist.append(w+s)
        wordlist.append(w.capitalize()+s)

with open("wordlist.txt", "w") as f:
    for word in wordlist:
        f.write(word + "\n")

print("Wordlist Generated Successfully")
```

Run Script

python3 dictionary_generator.py

```
┌──(sakshi㉿kali)-[~/Desktop/Dictionary_Generator]
└─$ python3 dictionary_generator.py
Wordlist Generated Successfully
```

Generated Wordlist Example

cat wordlist.txt

```
┌──(sakshi㉿kali)-[~/Desktop/Dictionary_Generator]
└─$ ls
dictionary_generator.py  wordlist.txt

┌──(sakshi㉿kali)-[~/Desktop/Dictionary_Generator]
└─$ cat wordlist.txt
sakshi@
Sakshi@
sakshi#
Sakshi#
sakshi123
Sakshi123
sakshi!
Sakshi!
sakshi
Sakshi
2001@
2001@
2001#
2001#
2001123
2001123
2001!
```

STEP 3 — Hash Extraction (Linux Shadow)

• Linux stores password hashes in:
/etc/shadow

• Gain Root Access: Elevate privileges to read restricted files.
sudo –s

```
┌──(sakshi㉿kali)-[~/Desktop/Hash_Extraction]
└─$ sudo -s
```

• Copy Necessary Files: Create a backup of passwd and shadow in your current directory.
cp /etc/passwd .
cp /etc/shadow .

```
┌──(root㉿kali)-[/home/sakshi/Desktop/Hash_Extraction]
└─# cp /etc/passwd .

┌──(root㉿kali)-[/home/sakshi/Desktop/Hash_Extraction]
└─# cp /etc/shadow .
```

• Combine Hashes (Unshadow): Use the unshadow utility (part of John the Ripper) to combine the files, creating a format recognized by password crackers.
unshadow passwd shadow > myhashes.txt

```
┌──(root㉿kali)-[/home/sakshi/Desktop/Hash_Extraction]
└─# unshadow passwd shadow > myhashes.txt

Created directory: /root/.john
```

• Verify Extraction: Display the extracted hashes.
cat myhashes.txt

```
┌──(root㉿kali)-[/home/sakshi/Desktop/Hash_Extraction]
└─# cat myhashes.txt
Debian-gdm:!:130:134:Gnome Display Manager:/var/lib/gdm3:/bin/false
sakshi:$y$j9T$1SgGQU6WXy.xhGI35IRr61$wTXodwCXDjdQmzeorZa6hDTSTQJ5yv/fafPztjfSDU9:1000:1000:Sakshi Nalawade,,,:/home/sakshi:/usr/bin/zsh
snort:!:131:136:Snort IDS:/var/log/snort:/usr/sbin/nologin
```

Explanation:
$6 = SHA512
$5 = SHA256
$1 = MD5

STEP 4 — Extract Windows SAM Hashes (Offline)

From Windows system export registry: SAM, SYSTEM

Command:
• Displays all user accounts on the computer: Use the net user command is a Windows command used to view and manage user accounts, including creating, modifying, and deleting users.
net user

```
C:\Windows\System32>net user

User accounts for \\DESKTOP-SCPML7Q

-------------------------------------------------------------------------------
Admin                    Administrator            DefaultAccount
Guest                    WDAGUtilityAccount
The command completed successfully.
```

• Shows details of the Administrator account: This is used to view or manage the built-in Administrator account in Windows.
net user Administrator

```
C:\Windows\System32>net user administrator
User name                    Administrator
Full Name
Comment                      Built-in account for administering the computer/domain
User's comment
Country/region code          000 (System Default)
Account active               Yes
Account expires              Never

Password last set            18-03-2026 10:18:05
Password expires             Never
Password changeable          18-03-2026 10:18:05
Password required            Yes
User may change password     Yes

Workstations allowed         All
Logon script
User profile
Home directory
Last logon                   18-03-2026 16:49:03

Logon hours allowed          All

Local Group Memberships      *Administrators
Global Group memberships     *None
The command completed successfully.
```

To check Windows registry files: SAM, SYSTEM and SECURITY.
• In Windows usually refers to the system configuration folder that stores registry files like SAM and SYSTEM.

config

```
C:\Windows\System32\config>dir
 Volume in drive C has no label.
 Volume Serial Number is B0B4-6D28

 Directory of C:\Windows\System32\config

14-03-2026  19:54    <DIR>          .
26-02-2026  21:38    <DIR>          ..
17-03-2026  14:34         1,310,720 BBI
19-09-2025  12:50    <DIR>          bbimigrate
19-09-2025  12:50            28,672 BCD-Template
12-12-2024  20:22    <DIR>          BFS
15-03-2026  22:09        65,011,712 COMPONENTS
17-03-2026  14:34         5,505,024 DEFAULT
17-03-2026  11:38        10,518,528 DRIVERS
01-04-2024  12:51             8,192 ELAM
01-04-2024  12:56    <DIR>          Journal
01-04-2024  12:56    <DIR>          RegBack
17-03-2026  14:34           131,072 SAM
17-03-2026  14:34            65,536 SECURITY
17-03-2026  14:34       145,227,776 SOFTWARE
17-03-2026  14:34        29,884,416 SYSTEM
01-04-2024  12:56    <DIR>          systemprofile
15-02-2026  17:19    <DIR>          TxR
17-09-2025  09:36             8,192 userdiff
              11 File(s)    257,699,840 bytes
               8 Dir(s)  88,203,177,984 bytes free
```

Save (export) the registry hive into a file named : sam.save and system.save .

- sam.save: is used to export the SAM registry hive containing user password hashes for offline analysis.

reg save HKLM\SAM sam.save

- system.save: exports the SYSTEM registry hive, which contains keys required to decrypt password hashes from the SAM hive.

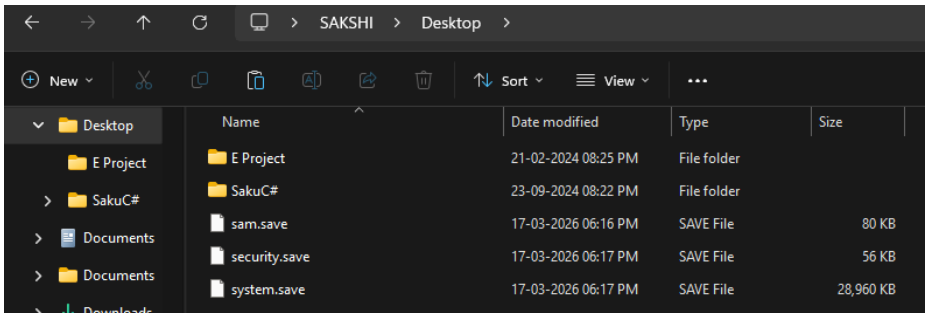
reg save HKLM\SYSTEM system.save

```
C:\Users\Admin>cd Desktop

C:\Users\Admin\Desktop>reg save hklm\sam sam.save
The operation completed successfully.

C:\Users\Admin\Desktop>reg save hklm\security security.save
The operation completed successfully.

C:\Users\Admin\Desktop>reg save hklm\system system.save
The operation completed successfully.
```



Copy files to Kali.
Send files as is to Kali Linux on Windows.

• File inside Information: Displays the contents of the SAM file, but since it is a binary registry hive, the output is not readable.
cat sam.save

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ ls
sam.save  security.save  system.save
┌──(sakshi㉿kali)-[~/Desktop]
└─$ cat sam.save
:◆◆◆rmtm◆◆&◆◆◆OfRg◆\System32\Config\SAM◆8◆*◆◆◆◆M
                    ~◆hbin◆◆◆◆nk,◆1◆◆(◆◆◆◆◆◆◆◆◆◆◆x◆◆◆◆ROO◆◆◆sk ◆◆◆◆◆◆◆ !?  ?
?8
◆1◆?l◆cL<◆Pi
                    ◆b◆~z◆◆◆◆◆◆◆
                    ◆◆◆◆nk ◆z5[◆(◆ x◆◆◆◆◆◆◆◆◆◆&◆SAM◆◆◆◆nk ◆"4◆◆(◆p◆Q◆◆
◆◆◆◆◆◆◆◆Domains◆◆◆◆◆◆◆◆◆◆◆nk AG◆◆◆0◆◆◆'◆◆◆◆(Q◆◆◆◆Account?◆◆◆◆lh◆◆|◆◆◆◆lh◆&X◆=
◆◆◆◆@◆◆◆◆◆
◆◆◆◆◆◆◆◆lhp◆◆◆◆◆◆skxxod◆HX4?  ◆◆◆◆nk ◆1◆◆(◆0 ◆  ◆◆◆◆◆#◆◆◆◆◆Aliases◆◆◆◆nk ◆1
◆◆(◆P◆◆◆◆◆◆◆◆◆◆◆◆◆◆000003EA◆◆◆◆vk◆  C◆◆◆a(◆P◆◆◆◆◆◆TX◆◆◆D0◆◆◆◆◆
                                                    $8
(g!>y◆]y◆◆)'V◆ f◆◆h◆◆◆◆  Debugger UsersDebugger Users are non administrat
ors who are allowed to use Visual Studio to debug processes, both locally and
 remotely. Only trusted users should be added to this group◆◆◆◆◆◆U◆
◆◆◆◆◆nk ◆1◆◆(◆P◆◆◆◆◆◆◆◆◆◆◆◆◆000003EB◆◆◆◆◆◆◆◆◆
◆◆◆◆ ◆◆◆◆vk`C◆◆◆◆◆◆◆`Hn◆  ◆◆◆D0◆◆◆◆◆
                                                    $8
```

samdump2 Tool : is a tool used in **Kali Linux** to extract **Windows user password hashes** from the SAM and SYSTEM files.

• To Displays usernames: Extracts Windows password hashes from SAM & SYSTEM files
samdump2 system.save sam.save

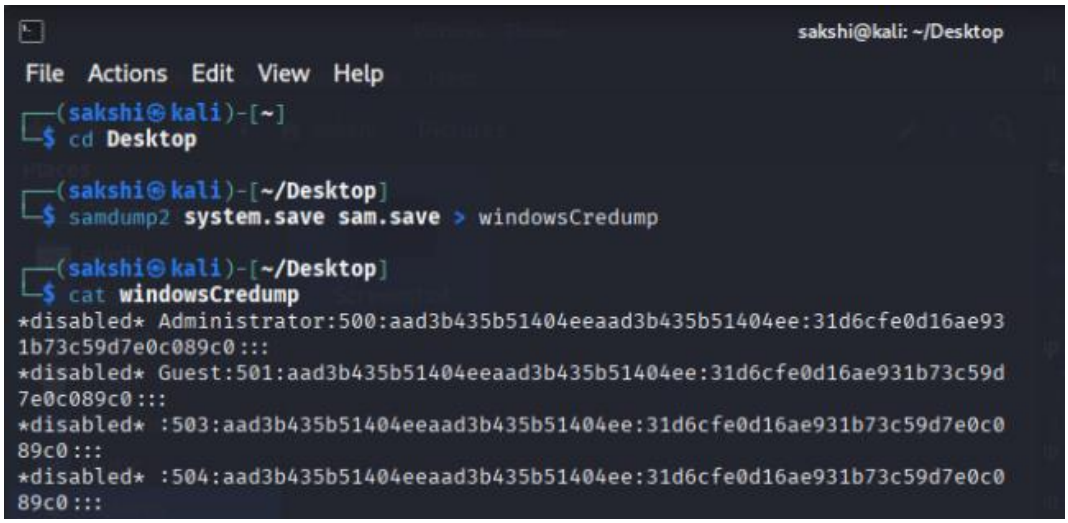
```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ samdump2 -h
samdump2 3.0.0 by Objectif Securite (http://www.objectif-securite.ch)
original author: ncuomo@studenti.unina.it

Usage: samdump2 [OPTION]... SYSTEM_FILE SAM_FILE
Retrieves syskey and extract hashes from Windows 2k/NT/XP/Vista SAM

  -d            enable debugging
  -h            display this information
  -o file       write output to file

┌──(sakshi㉿kali)-[~/Desktop]
└─$ samdump2 system.save sam.save
*disabled* Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae93
1b73c59d7e0c089c0:::
*disabled* Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d
7e0c089c0:::
*disabled* :503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c0
89c0:::
*disabled* :504:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c0
89c0:::
Admin:1001:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:
::
```

• Extracts password hashes and saves them into windowsCredump file.
Samdump2 system.save sam.save > windowsCredump
• Shows stored credential/hash data from the windowsCredump file.
cat windowsCredump

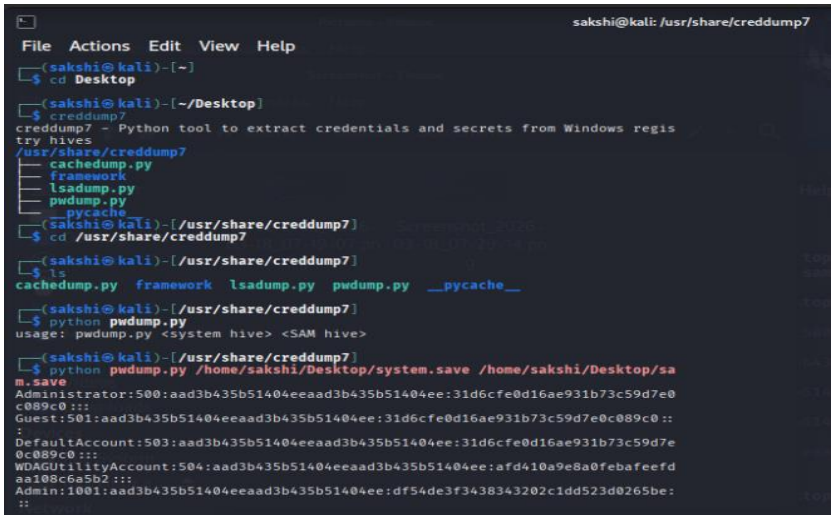


Creddump7 Tool: creddump7 is a tool used to extract Windows credentials and password hashes from offline registry files.

• Executes pwdump script to dump Windows user password hashes.
Python pwdump.py

• Combines SYSTEM + SAM files to decrypt and display Windows user password hashes (NTLM)
python pwdump.py /home/sakshi/Desktop/system.save /home/sakshi/Desktop/sam.save

• Extracts password hashes and saves them into windump file.
python pwdump.py /home/sakshi/Desktop/system.save /home/sakshi/Desktop/sam.save > windump



• Shows stored credential/hash data from the windump file.
cat windump

```
┌──(sakshi㉿kali)-[~]
└─$ cd Desktop

┌──(sakshi㉿kali)-[~/Desktop]
└─$ cat windump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0
c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0::
:
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e
0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:afd410a9e8a0febafeefd
aa108c6a5b2:::
Admin:1001:aad3b435b51404eeaad3b435b51404ee:df54de3f3438343202c1dd523d0265be:
::
```

hashcat Tool: is a powerful tool used to **crack password hashes** using GPU/CPU.

• Takes hash values (like NTLM, MD5, and SHA)
• Uses attacks like:
- Dictionary attack
- Brute-force attack
- Rule-based attack

• Displays detailed list of all files (including hidden) in the current directory.
ls -la

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ ls -la
total 29112
drwxr-xr-x  2 sakshi sakshi     4096 Mar 18 06:55 .
drwx------ 16 sakshi sakshi     4096 Mar 19 23:45 ..
-rw-------  1 sakshi sakshi    81920 Mar 17 07:46 sam.save
-rwxrw-rw-  1 sakshi sakshi    57344 Mar 17 07:47 security.save
-rwxrw-rw-  1 sakshi sakshi 29655040 Mar 17 07:47 system.save
-rw-rw-r--  1 sakshi sakshi      440 Mar 18 06:51 windowsCredump
-rw-rw-r--  1 sakshi sakshi      426 Mar 18 06:55 windump
```

• Uses a dictionary attack to **crack NTLM password hashes** from the windowsCredump file using the rockyou wordlist.

hashcat –a 0 –m 1000 windowsCredump /usr/share/wordlist/rockyou.txt

• hashcat → password cracking tool
• -a 0 → dictionary attack
• -m 1000 → NTLM hash type
• windowsCredump → file containing hashes
• /usr/share/wordlists/rockyou.txt → wordlist used to guess passwords

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ hashcat -a 0 -m 1000 windowsCredump /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl pr
oject]
====================================================================================================================================
* Device #1: cpu-haswell-Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz, 1424/2912 MB (512 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 5 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 0 MB

Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14344385
* Runtime...: 3 secs

31d6cfe0d16ae931b73c59d7e0c089c0:

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 1000 (NTLM)
Hash.Target......: 31d6cfe0d16ae931b73c59d7e0c089c0
Time.Started.....: Thu Mar 19 23:54:14 2026 (0 secs)
Time.Estimated...: Thu Mar 19 23:54:14 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:    25340 H/s (0.13ms) @ Accel:256 Loops:1 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 5120/14344385 (0.04%)
Rejected.........: 0/5120 (0.00%)
Restore.Point....: 4608/14344385 (0.03%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: Liverpool → babygrl
Hardware.Mon.#1..: Util: 54%

Started: Thu Mar 19 23:52:43 2026
Stopped: Thu Mar 19 23:54:16 2026
```

- Loads NTLM hashes (with usernames) from the file for cracking

hashcat –m 1000 –username windowsCredump

- --username → tells hashcat the file contains **username:hash format**

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ hashcat -m 1000 --show --username windowsCredump
*disabled* Administrator:31d6cfe0d16ae931b73c59d7e0c089c0:
*disabled* Guest:31d6cfe0d16ae931b73c59d7e0c089c0:
*disabled* :31d6cfe0d16ae931b73c59d7e0c089c0:
*disabled* :31d6cfe0d16ae931b73c59d7e0c089c0:
Admin:31d6cfe0d16ae931b73c59d7e0c089c0:
```

- Uses a dictionary attack to **crack NTLM password hashes** from the winddump file using the rockyou wordlist.

hashcat –a 0 –m 1000 windump /usr/share/wordlist/rockyou.txt

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ hashcat -a 0 -m 1000 windump /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
====================================================================================================================================
* Device #1: cpu-haswell-Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz, 1424/2912 MB (512 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 5 digests; 3 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

INFO: Removed hash found as potfile entry.

Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

Approaching final keyspace - workload adjusted.

Session..........: hashcat
Status...........: Exhausted
Hash.Mode........: 1000 (NTLM)
Hash.Target......: windump
Time.Started.....: Fri Mar 20 00:00:14 2026 (23 secs)
Time.Estimated...: Fri Mar 20 00:00:37 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:   616.4 kH/s (0.22ms) @ Accel:256 Loops:1 Thr:1 Vec:8
Recovered........: 2/3 (66.67%) Digests (total), 1/3 (33.33%) Digests (new)
Progress.........: 14344385/14344385 (100.00%)
Rejected.........: 0/14344385 (0.00%)
Restore.Point....: 14344385/14344385 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: $HEX[206b72697374656e616e6e65] → $HEX[042a0337c2a156616d6f732103]
Hardware.Mon.#1..: Util: 51%

Started: Fri Mar 20 00:00:08 2026
Stopped: Fri Mar 20 00:00:39 2026
```

- Loads NTLM hashes (with usernames) from the file for cracking

hashcat –m 1000 –username windump

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ hashcat -m 1000 --show --username windump
Administrator:31d6cfe0d16ae931b73c59d7e0c089c0:
Guest:31d6cfe0d16ae931b73c59d7e0c089c0:
DefaultAccount:31d6cfe0d16ae931b73c59d7e0c089c0:
Admin:df54de3f3438343202c1dd523d0265be:321
```

STEP 5 — Dictionary Attack Simulation

Use **John the Ripper**.

Command:
john –format=NT --wordlist=wordlist.txt windump

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ john --format=NT --wordlist=wordlist.txt windump
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (NT [MD4 256/256 AVX2 8×3])
Remaining 2 password hashes with no different salts
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 8 candidates left, minimum 24 needed for performance.
0g 0:00:00:00 DONE (2026-03-21 13:34) 0g/s 800.0p/s 800.0c/s 1600C/s 321
Session completed.
```

Check cracked passwords:
john --show –format=NT windump

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ john --show --format=NT windump
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c
089c0:::
Admin:321:1001:aad3b435b51404eeaad3b435b51404ee:df54de3f3438343202c1dd523d0265be
:::

3 password hashes cracked, 2 left
```

STEP 6 — Password Strength Analyser

Create Python script.
File: password_strength.py

```
sakshi@kali: ~/Desktop
File  Actions  Edit  View  Help
  GNU nano 8.4                                password_strength.py
import math

password = input("Enter Password: ")

charset = 0

if any(c.islower() for c in password):
    charset += 26
if any(c.isupper() for c in password):
    charset += 26
if any(c.isdigit() for c in password):
    charset += 10
if any(c in "!@#$%^&*" for c in password):
    charset += 10

entropy = len(password) * math.log2(charset)

print("Password Entropy:", entropy)

if entropy < 40:
    print("Weak Password")
elif entropy < 60:
    print("Medium Strength")
else:
    print("Strong Password")
```

To check password pattern
Python3 password_strength.py
Run this command and then enter the password:_____
To get password information such as password Entropy and password type.

```
┌──(sakshi㉿kali)-[~/Desktop]
└─$ python3 password_strength.py
Enter Password: 321
Password Entropy: 9.965784284662087
Weak Password
```

STEP 7 — Generate Security Audit Report

Report Generation

After performing password cracking and analysis, a detailed report is generated to summarize findings, risks, and recommendations.

1 Identified Weak Passwords

The system lists all passwords that were successfully cracked during the attack simulation.
Examples:
* 321
* admin123
* password
* user@123

These passwords are considered weak due to their simplicity and predictability.

2 Cracking Time Analysis

The time required to crack each password is recorded to evaluate its strength.

Password	Attack Type	Time Taken
321	Dictionary Attack	1 second
admin123	Dictionary Attack	2–5 seconds
Sakshi@123	Brute Force	Few minutes
Xy#9Lp!Q2$	Brute Force	Not cracked

This shows that simpler passwords are cracked much faster than complex ones.

6.3 Risk Assessment

Based on the results, the following risks are identified:
* Weak passwords can be cracked instantly.
* Reused passwords increase vulnerability.
* Predictable patterns (names, dates) are easy targets.
* Systems without password policies are at high risk.

6.4 Mitigation Strategies

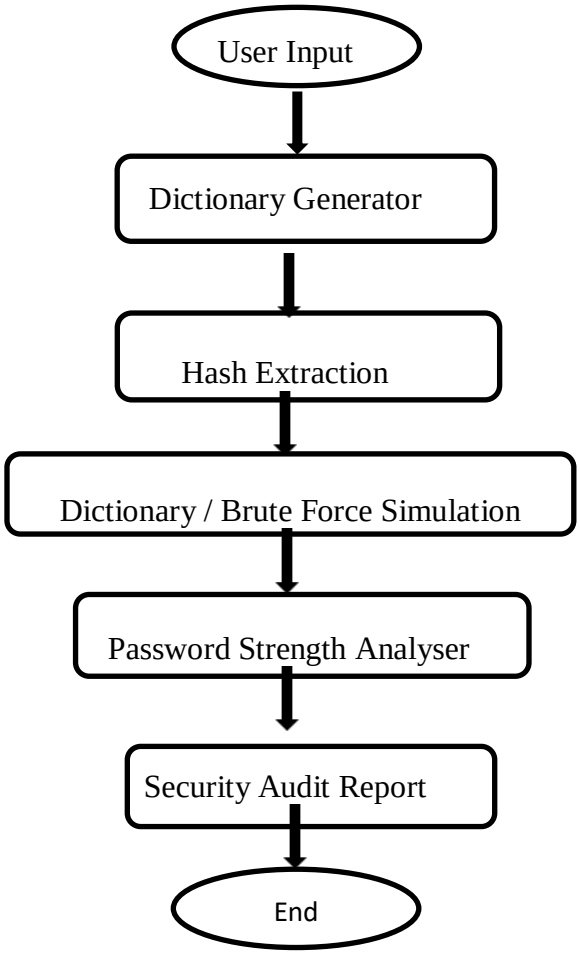
To improve security, the following measures are recommended:
* Use strong passwords (minimum 8–12 characters).
* Include uppercase, lowercase, numbers, and symbols.
* Avoid common words and personal information.
* Implement account lockout mechanisms.
* Enable multi-factor authentication (MFA).
* Regularly update passwords.

6.5 Summary

The report highlights that password strength directly impacts system security. Strong and complex passwords significantly reduce the chances of successful attacks.

Proper awareness and security practices can prevent most credential-based attacks.

Final Project Architecture



Learning Outcome

This project enables understanding of:

- How passwords are stored, hashed, and protected
- Ethical password cracking methodologies
- Authentication security auditing
- Red team vs blue team password assessment techniques
- Best practices for secure password creation and enforcement
- Defensive password policies